

Functional Size Measurement in Agile Development

Velocity in Agile Sprints

Thomas Fehlmann

Euro Project Office AG, Zurich

thomas.fehlmann@e-p-o.com

Andrea Gelli

QSMA AG – QSM Switzerland, Zurich

andrea.gelli@qisma.ch

ABSTRACT

Agile teams measure their velocity for performance, based on Story Points. However, such velocity does not allow predicting when the product will be finished. Story points measure effort only. They do not discriminate between creating functionality and other tasks. Non-functional requirements (NFR), such as enhancing product quality, test coverage, removing technical debt, as well as process-related NFR such as agreeing with stakeholders, getting requirements right, or documenting, consume effort but do not add functionality. Thus, it remains unclear whether the product makes any progress, or the team is just looping around. Euro Project Office has therefore developed a method how to complement a product backlog by functional size, indicating progress and completeness in unambiguous terms. The method is based on the international standard ISO/IEC 14143 and ISO/IEC 19761. NFR are understood as in . Tools are available as open source and can be used by development teams with minimum investment into training.

CCS CONCEPTS

• **Software and its engineering**; • **Software creation and management**; • **Software development process management**; • **Software development methods**; • **Agile software development**

KEYWORDS

Agile Development, Technical Debt, Functional Size, Product Backlog, Continuous Delivery

ACM Reference Format:

Thomas Fehlmann and Andrea Gelli. 2023. Functional Size Measurement in Agile Development: Velocity in Agile Sprints. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE '23)*, June 14–16, 2023, Oulu, Finland. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3593434.3593486>

1 INTRODUCTION

Today, organizations use the agile development paradigm successfully for product development as well as for supporting operations. The authors' experience area includes, but is not limited to, government applications that involve dozens of team members, years

of product development, different stakeholders, and requirements that change as fast as legislation does.

The teams that the authors accompanied during their agile journeys put testing into user stories, just as design, documentation, or getting requirements right from stakeholders. Functional user requirements (FUR) and NFR are in their sum equivalent to all user stories and describe the total effort that agile teams assess with story points. There is no user story that is neither FUR nor NFR. We use the joint IFPUG-COSMIC glossary to distinguish between FUR, product-related NFR (quality), and process-related NFR [3].

In a technical environment, software engineering refers to software designed, programmed, and tested to meet specifications. This is not the environment that we encountered. Especially in public environments, requirements change whenever legislation changes or when some emergency dictates it. It is similar for banking. Thus, agile development teams better avoid upfront investments for commercial and administrative environments. The real challenge is to get requirements right during development. This is easier and cheaper when stakeholders have early access to working products, even if buggy and incomplete. Security can be built in from the beginning; safety in the sense of ISO 26262 [4] is most often not a concern. Thus, missing specifications are not a concern.

Also, not much of a concern are early cost estimations. Without an initial requirements specification, it is difficult to estimate the cost of developing a product [5]. In most cases, the organizations mentioned allocate a development team for a certain time without precisely knowing what they will deliver. Time is the constraint; cost overruns are controlled by cutting expectations. Traditional software development teams often delivered nothing because the requirements specification was never finished due to stakeholder disagreement. Agile teams at least deliver something. One can cost estimate an initial backlog, if available. However, only a small part of the initial user stories conforms to the *Definition of Ready* and ever make it into a sprint [6, p. 126]. Instead, new user stories appear, become prominent, and get implemented. Others disappear from the backlog or stay there forever, never to be touched.

This observation is also backed by ISBSG benchmark repository data, see for instance [7]. Not only is it important how many function points are implemented, but it is even more important to know which functionality is implemented and which is not.

In such environments it is difficult to rely on backlogs measured by story points alone. Even if technical debt is monitored and considered, recognizing the end of new development and entrance into maintenance phase is hard to identify. This leads to prolonged development phases and to higher upfront capital investment. Switching into maintenance mode, and probably adopting a DevOps paradigm, could correctly allocate cost to operations in time. Thus, the decreasing amount of new added functionality is a good indication for

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EASE '23, June 14–16, 2023, Oulu, Finland

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0044-6/23/06...\$15.00

<https://doi.org/10.1145/3593434.3593486>

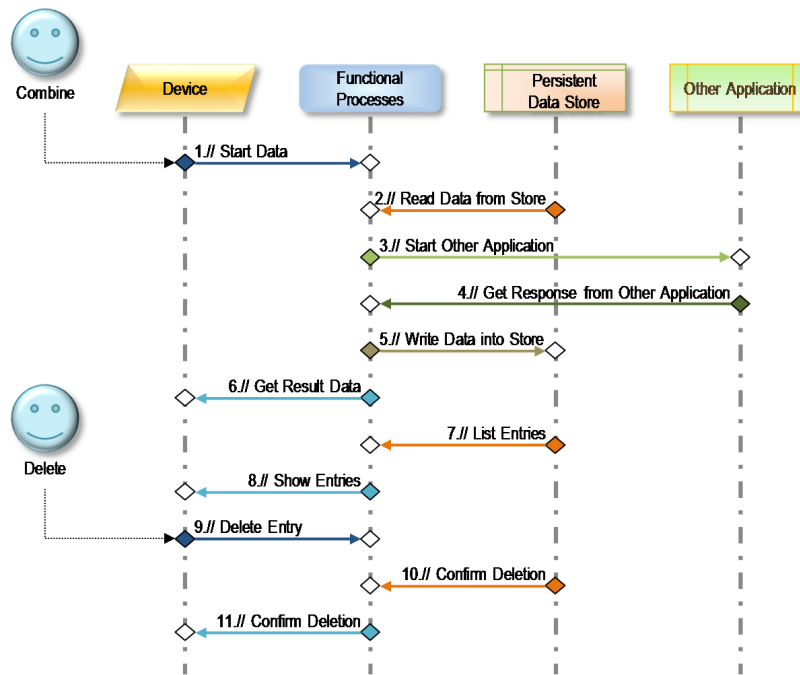


Figure 1: Data Movement Map According to COSMIC

when a product should shift from agile development into DevOps. Or, in conventional terms, when the product is “finished”.

2 METHOD

Functional size measurement (FSM) has a long history and is well established in traditional software development environments, especially in government and administration. It has been used to predict the cost of initial development, of change requests, and to determine the end of development. However, with the agile paradigm, without a detailed upfront requirements specification, new functionality appears whenever the problem area gets better understood by the development team.

FSM delivers a model of the system under development than can be compared using benchmarking database such as ISBSG’s [8] and QSM’s SLIM – Metrics [9]. It yields cost and effort predictions based on functional size, adjusting for team size, and taking industry, safety, and security requirements into account.

2.1 Modeling with ISO/IEC 19761 COSMIC

According to ISO/IEC 14143, requirements shall be modeled up to the desired granularity, conforming to the needs of the development and testing team. If the desired granularity is up to microservices, the Kubernetes orchestrating script already delivers the model needed. For networks and heavily distributed applications, the system architecture might indicate the needed granularity.

There are various FSM methods that are conformant to ISO/IEC 14143. According to the ISO/IEC 19761 COSMIC model, size is defined as the number of data movements between objects of interest, and thus easily countable [2]. The model identifies all elements that

are important for assessing functionality within the granularity defined and serves as a means of communication between the team and other stakeholders. The Data Movement Map (Figure 1) identifies all objects of interest including functional processes, devices and peer applications, persistent store, triggers, and data groups moved by data movements. They look like the well-known UML sequence diagrams, but without their full complexity.

Because developers hate FSM – too many managers have been led by the misconception that functional sizing can be used to measure individual performance, and agile teams mistakenly view FSM as a wasted upfront investment – the authors prefer the term “modeling of functionality” to anything that has to do with measurement, taking advantage from the obvious fact that models are used for measurement, whether in house construction or building construction. But avoiding bad associations is an important part of human relationship management.

In practice, the data movement map (Figure 1) is a convenient tool to introduce FSM as a modeling technique that is useful for communication within the team; sometimes also with other stakeholders. Especially for a microservice architecture approach, the models are highly appreciated. When a team has a few dozen, or hundreds, of microservices, the models are extremely helpful to recall what these services are expecting, doing, and delivering.

2.2 The Ideal Case

Ideally, there is an initial vision that can be modeled and sized with ISO/IEC 19761 COSMIC, usually named *Initial Backlog*.

This is a good start but remember that the functionality that finally gets implemented diverges quite a bit. You can at least measure progress against that initial backlog.

2.3 The Real Case

But even in reality, where no vision is available or requirements too much blurred, you can start sizing the implemented software as soon as a first sprint backlog is available that meets the definition of ready. You mark or create the implemented data movements in the model and count them. This yields the functional size of the first sprint. Now you can compare it with the story points estimated per sprint.

From sprint to sprint, you now extend the model by the additional data movements and its corresponding objects. However, some of the functionality implemented before will have to be disregarded or modified. While this adds to sprint performance, the overall size of the product will not grow accordingly. There is always a loss. This loss function is of high interest, especially for product owners and the development team, because it indicates the amount of technical debt and misunderstood requirements.

Because agile development never ends – agile is for product development, it is not a project methodology, agile development has no defined end – you cannot determine when the product is finished. However, by comparing new functionality added with modified functionality (product size growth rate) and by comparing the amount of story points used for adding functionality with story points used for NFR, i.e., the FUR/NFR ratio, one sees the product maturity. While new functionality keeps being added, the needs of the users are not yet fulfilled. New requirements pop up while working on the product. This is expected, welcomed, and explains the advantages of agile development.

Once product size growth stops and the FUR/NFR ratio drops, there is a strong presumption that new feature development is complete. There might still be an important backlog of user stories referring to testing and removing technical debt, but once this is done, the product is entering maintenance phase and funding the effort should no longer be based on capital expenditure.

Obviously, one would like to know in advance when this is likely to happen, because of allocating enough but not too many sprints to product development. There exist many attempts to do so, see for instance the meta-study from Hoda et. al. [10]. The seminal paper on this topic is also available as an ISBSG resource [11]. Newer attempts try using *Artificial Intelligence* (AI) for prediction; however, it is difficult to get valid training sets for such an AI model. A more reasonable approach should take into consideration the reasons that lead to detection of missing functionality. The main reason is usually that the implemented product functionality does not meet the users' value expectations. To counteract this, it is necessary to capture and analyze these values, preferably in the beginning as an upfront investment, and then develop the product based on this knowledge. Six Sigma transfer functions are suitable for the conversion into product functionality. The authors have published on this [12]. There is an international standard ISO 16355 that explains the method [13].

3 FINDINGS AND RESULTS

As consultants and project office, we accompanied several software developments during the last seven years and found that:

- Comparing story points with functional size allows a much more accurate progress indicator than looking at the backlog alone.
- The variation between story points and functional progress shows obstacles of various sorts that can cause delays and frustration among the stakeholders.
- The effort to look at functional size per sprint is minimal if a suitable tool is used. The Euro Project tool is based on MS Excel and available with minimal training.
- Modeling your software ideas and achievements is as helpful for software developers as for building engineers and architects. It supports communication within the team and with stakeholders.

As an example, we show the observations made for a relatively simple mobile app development project that ran into a security problem during sprint no. 4. As a result, some functionality had to be re-developed. Details can be found in Fehlmann & Kranich [14].

Figure 2 shows that product growth stalled when encountering a security issue in sprint 4, and again when entering final testing in sprint 7.

Meanwhile the sprint productivity remained stable (Figure 3):

Figure 4 provides explanations. In our sample app development project, the sprint productivity was affected by the security issue already mentioned in sprint 4. Moreover, no functionality was developed in sprint 5 because work was fully dedicated to removing technical debt, and final testing.

4 CONCLUSION

Story points and FSM are complementary. While story points reflect the teams' belief in what effort they can provide, functionality is what interests the sponsor usually most. Velocity in terms of functionality is often adversely impacted by quality, especially by the technical debt accumulated during development.

On the other hand, agile development often delivers higher quality than traditional software development. The reason for this can be made visible by comparing story points with delivered functionality.

Agile development methods must consider that requirements elicitation happens typically by presenting some initial functionality to the stakeholders, getting their feedback, and modifying functionality accordingly. Thus, part of the decreasing product size growth (Figure 3) originates from wrong, or unclear, requirements, that need revision. The security problem affecting sprint no. 4 is just one such example that might have escaped to traditional software development when not made explicit by specifications. In our sample project, the problem was caused by a smartphone camera used for scanning paper bills, whose driver uploaded all images taken immediately to the Internet, without asking. Only extensive testing after development eventually could detect such a privacy leak while every young Instagram user knows about and possibly uses this functionality. Thus, involving many diverse stakeholders in earlier phases is an advantage compared to traditional development approaches.

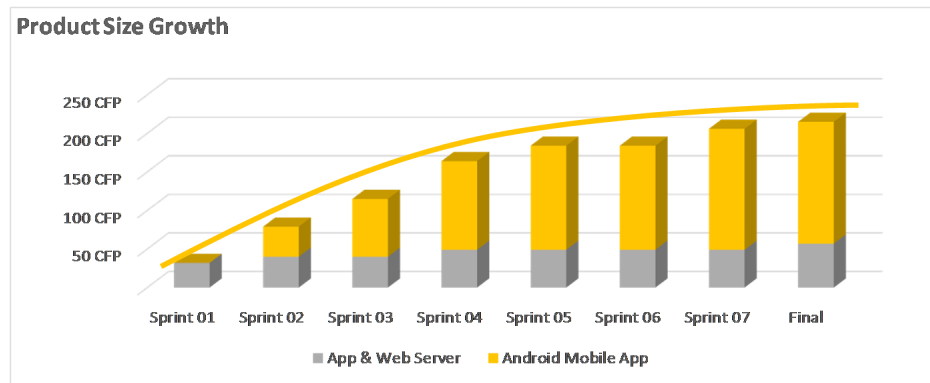


Figure 2: Product size growth over seven sprints

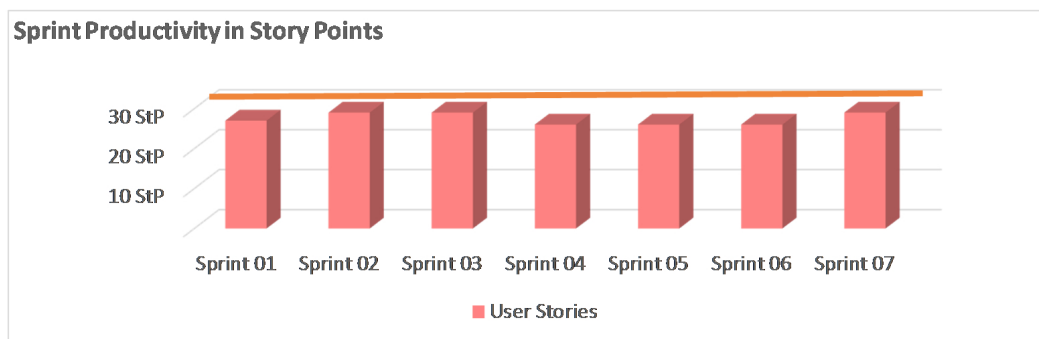


Figure 3: Sprint productivity remained stable

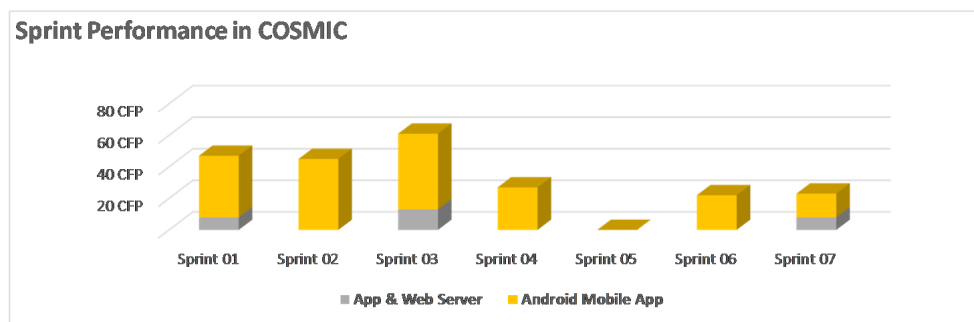


Figure 4: Functional size with irregular growth per sprint.

The details of the presented case study are available in [14].

In environments such as banking and public administration, where process NFR are particularly demanding, sizing functionality per sprint aside of story points does help agile teams in benchmark situations. Usually, these teams are not performing worse than others, but the sheer amount of organization-dependent process constraints is a strong argument. Getting requirements right is a very tedious task. Using the agile development paradigm is paramount for success. With this, the true cost of requirements elicitation become measurable and even plannable.

When developing software for highly complicated customer, such as the EU with 27 member states, or Switzerland with 26 cantons and many different languages, there is no alternative to sizing functionality. Effort and story points alone are not enough.

Even for safety-critical software, agile approaches are considered and being incorporated into new approaches. For instance, when integrating ISO 26262 SOTIF (Safety of The Intended Functionality) with agile systems engineering, data movement maps, or similar, play a decisive role. With FSM in Agile, these models already exist for free and might be used for overlaying SOTIF with the systems engineering pipeline [15].

Monitoring the functional size performance per sprint yields more valuable information for the development team than just monitoring a burndown chart, referring to tasks instead of size. It allows a more precise assessment of development progress and helps with safety and security [16]. Moreover, if the functional growth nears zero, it means that the product has entered maintenance mode and it should no longer be regarded as a development project. The DevOps paradigm would then become more appropriate.

REFERENCES

- [1] ISO/IEC 14143, "Information technology - Software measurement - Functional size measurement - Part 1: Definition of concepts," ISO/IEC JTC 1/SC 7, Geneva, Switzerland, 2019.
- [2] ISO/IEC 19761, "Software engineering - COSMIC: a functional size measurement method," ISO/IEC JTC 1/SC 7, Geneva, Switzerland, 2019.
- [3] COSMIC and IFPUG, "Glossary of terms for Non-Functional Requirements and Project Requirements used in software project performance measurement, benchmarking and estimating," September 2015. [Online]. Available: <http://cosmic-sizing.org/publications/glossary-of-terms-for-nf-and-project-requirements/>. [Accessed 13 April 2023].
- [4] ISO 26262-1, "Road vehicles - Functional Safety - Part 1: Vocabulary," ISO/TC 22/SC3, Geneva, 2011.
- [5] T. Hacaloglu and O. Demirs, "Measureability of functional size in Agile software projects - Multiple Case Studies with COSMIC FSM," in 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), Kallithea, Greece, 2019.
- [6] B. Meyer, *Agile! - The Good, the Hype and the Ugly*, Springer International Publishing Switzerland, 2014.
- [7] H. v. Heeringen, "Analysis of Velocity (Delivery Speed)," ISBSG Short Papers, Melbourne, Victoria, Australia, 2023.
- [8] P. Hill, Ed., *Practical Software Project Estimation* 3rd Edition, New York, NY: McGraw-Hill, 2010.
- [9] Angela Maria Lungu, Editor, *QSM Software Almanac*, McLean, VA: Quantitative Software Management, Inc., 2019.
- [10] R. Hoda, N. Salleh, J. Grundy and H. M. Tee, "Systematic literature reviews in agile software development," *Information and Software Technology*, 2017.
- [11] C. Commeyne, A. Abran and R. Djouab, "Effort Estimation with Story Points and COSMIC Function Points - An Industry Case Study," *Software Measurements News*, vol. 21, no. 1, pp. 25-36, 2016.
- [12] T. M. Fehlmann and E. Kranich, "Early Software Project Estimation the Six Sigma Way," *Lecture Notes in Business Information Processing*, vol. 199, pp. 193-208, 2014.
- [13] ISO 16355, "Applications of Statistical and Related Methods to New Technology and Product Development Process - Part 1: General Principles and Perspectives of Quality Function Deployment (QFD)," ISO TC 69/SC 8/WG 2 N 14, Geneva, Switzerland, 2015.
- [14] T. M. Fehlmann and E. Kranich, "Functional Size Measurement in Agile," in *ICSEA 2021*, Barcelona, 2021.
- [15] K. Post and C. K. Davey, "Integrating SOTIF and Agile Systems Engineering," *SAE Technical Paper*, Warrendale, PA, 2019.
- [16] T. M. Fehlmann and E. Kranich, "Theoretical Aspects of Consumer Metrics for Safety & Privacy," *Systems, Software and Services Process Improvement*, vol. 896, pp. 649-653, 2018.